

My Genetic Algorithm for Travelling Salesman Problem

Generated by Doxygen 1.16.1

1 My Genetic Algorithm for Travelling Salesman Problem	1
1.1 My Genetic Algorithm for Travelling Salesman Problem	1
1.1.1 Algorithm Overview	1
1.1.1.1 Initialization	1
1.1.1.2 Evolution Loop	1
1.1.2 Building	1
1.1.2.1 Compilation	1
1.1.2.2 Executing	2
1.1.3 Minimal Working Example	2
1.1.4 Author	2
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Chromosome Class Reference	7
4.1.1 Detailed Description	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 Chromosome() [1/3]	8
4.1.2.2 Chromosome() [2/3]	8
4.1.2.3 Chromosome() [3/3]	8
4.1.3 Member Function Documentation	8
4.1.3.1 check_valid_string()	8
4.1.3.2 get_string() [1/2]	9
4.1.3.3 get_string() [2/2]	9
4.1.3.4 L1()	9
4.1.3.5 mutate()	9
4.1.3.6 operator=()	10
4.1.3.7 overwrite_string()	10
4.1.3.8 recombine()	10
4.1.3.9 save()	11
4.2 Cities Class Reference	11
4.2.1 Detailed Description	11
4.2.2 Member Function Documentation	12
4.2.2.1 get_city()	12
4.2.2.2 get_distance()	12
4.2.2.3 Initialize_Cities_Circle()	12
4.2.2.4 Initialize_Cities_Square()	12
4.2.2.5 Initialize_From_File()	13
4.2.2.6 save_positions()	13

4.3 City Class Reference	13
4.3.1 Detailed Description	14
4.3.2 Constructor & Destructor Documentation	14
4.3.2.1 City()	14
4.4 Genetic Class Reference	14
4.4.1 Detailed Description	15
4.4.2 Constructor & Destructor Documentation	15
4.4.2.1 Genetic() [1/2]	15
4.4.2.2 Genetic() [2/2]	15
4.4.3 Member Function Documentation	16
4.4.3.1 average_L1()	16
4.4.3.2 best_L1()	16
4.4.3.3 best_L1_index()	16
4.4.3.4 calculate_fitness_first()	16
4.4.3.5 check_all_chromosomes()	17
4.4.3.6 get_chromosome()	17
4.4.3.7 new_generation()	17
4.4.3.8 overwrite_first_chromosome()	18
4.5 Position Class Reference	18
4.5.1 Detailed Description	18
4.5.2 Constructor & Destructor Documentation	18
4.5.2.1 Position() [1/2]	18
4.5.2.2 Position() [2/2]	19
4.5.3 Member Function Documentation	19
4.5.3.1 l1()	19
4.6 Random Class Reference	19
4.6.1 Detailed Description	20
5 File Documentation	21
5.1 include/chromosome.hpp File Reference	21
5.1.1 Detailed Description	21
5.2 chromosome.hpp	21
5.3 include/city.hpp File Reference	22
5.3.1 Detailed Description	22
5.4 city.hpp	22
5.5 include/genetic.hpp File Reference	23
5.5.1 Detailed Description	23
5.6 genetic.hpp	23
5.7 include/position.hpp File Reference	24
5.7.1 Detailed Description	24
5.8 position.hpp	24
5.9 include/random.h File Reference	24

5.9.1 Detailed Description	25
5.10 random.h	25
5.11 source/chromosome.cpp File Reference	25
5.11.1 Detailed Description	25
5.12 source/city.cpp File Reference	25
5.12.1 Detailed Description	26
5.13 source/genetic.cpp File Reference	26
5.13.1 Detailed Description	26
5.14 source/random.cpp File Reference	26
5.14.1 Detailed Description	26
Index	27

Chapter 1

My Genetic Algorithm for Travelling Salesman Problem

1.1 My Genetic Algorithm for Travelling Salesman Problem

This project aims to solve the travelling salesman problem (TSP) using an implementation of a genetic algorithm (GA), the genetic algorithm is implemented with parallelization.

1.1.1 Algorithm Overview

1.1.1.1 Initialization

Creates a collection of [Cities](#), one can choose to create it from a file or generate a random configuration. Then for each rank of the program generate a population with N individuals.

1.1.1.2 Evolution Loop

For each generation (and each rank):

1. Orders the individual in the generation from best to worst.
2. Until the generation has a full population:
 - (a) *Selection*: chooses 2 random parents to generate 2 offsprings
 - (b) *Crossover*: applies with a certain probability the crossover to the offsprings.
 - (c) *Mutations*: applies 4 mutations (swap, shift, permute, invert) each with a certain probability.
 - (d) Saves the offsprings to the new generation, until the new population is full.
3. Saves the best path to a file and the best path length and average path length to another file.

Every M generations a migration happens: the best individual of each rank is sent to another random one.

1.1.2 Building

1.1.2.1 Compilation

Inside the root directory:

```
make
```

Note: the compilation requires `mpicxx` and `c++20`.

1.1.2.2 Executing

```
mpirun -n 7 ./main.exe
```

You can modify the number of cores used at your liking.

1.1.3 Minimal Working Example

```
#include "genetic.hpp"
#include "random.h"
Random rnd; // setup the random number generator.
const unsigned int N_CITIES = 34;
const unsigned int N_INDIVIDUALS = 100;
const unsigned int N_GENERATIONS = 500;
// Create a random map on the unit circle.
auto cities_circle = Cities::Initialize_Cities_Circle(N_CITIES, 1., rnd);
// Create a random population of N_INDIVIDUALS
auto population = Genetic(N_INDIVIDUALS, cities, rnd);
// Compute the new generation N_GENERATIONS times
for (unsigned int i = 0; i < N_GENERATIONS; ++i)
    population.new_generation(cities, 2., rnd);
```

1.1.4 Author

Mattia Ballico

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chromosome	Represents a chromosome in the genetic algorithm	7
Cities	Collection of cities	11
City	Unique city	13
Genetic	Genetic algorithm for solving the traveling salesman problem	14
Position	Position in a 2D space	18
Random	A class for generating random numbers	19

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

include/ chromosome.hpp	Stores class Chromosome	21
include/ city.hpp	Stores the class City , and the class Cities	22
include/ genetic.hpp	Stores the class Genetic	23
include/ position.hpp	Stores the Position class	24
include/ random.h	This header file defines the Random class, which contains functions for generating random numbers	24
source/ chromosome.cpp	Implements the modules of the class Chromosome	25
source/ city.cpp	Implements the modules of the class City , and the class Cities	25
source/ genetic.cpp	Implementation of the Genetic class methods	26
source/ random.cpp	Implements the modules of the class Random	26

Chapter 4

Class Documentation

4.1 Chromosome Class Reference

Represents a chromosome in the genetic algorithm.

```
#include <chromosome.hpp>
```

Public Member Functions

- **Chromosome** (std::vector< unsigned int > possible_UIDs, **Random** &rnd)
Constructs a chromosome with a random route based on the provided possible UIDs. The constructor initializes the chromosome with a valid route that includes all the possible UIDs exactly once. The first element of the chromosome is set to the minimum UID to reduce symmetry of the problem.
- **Chromosome** (std::vector< unsigned int > possible_UIDs, std::vector< unsigned int > string)
Constructs a chromosome with a given route. The constructor checks if the provided route is valid (contains all possible UIDs exactly once and starts with the minimum UID).
- **Chromosome** (const **Chromosome** &other)
copy constructor
- **Chromosome** & **operator=** (const **Chromosome** &other)
copy assignment operator
- void **mutate** (const double &p_swap, const double &p_shift, const double &p_permute, const double &p_invert, **Random** &rnd)
mutates the chromosome by applying various mutation operators with given probabilities Mutation operators include swap, shift, permute, and invert, which are applied sequentially.
- bool **check_valid_string** (std::vector< unsigned int > possible_UIDs) const
checks if the chromosome is valid. A chromosome is valid if it contains all possible UIDs exactly once and starts with the minimum UID.
- double **L1** (const **Cities** &map) const
calculates the total distance of the route represented by the chromosome.
- std::vector< unsigned int > **get_string** () const
gets the route represented by the chromosome.
- void **save** (std::ofstream &fout) const
saves the route represented by the chromosome to a file. The route is saved as a sequence of UIDs, one per line.
- void **overwrite_string** (std::vector< unsigned int > new_string)
sets the chromosome to a new given route
- std::vector< unsigned int > & **get_string** ()
gets the route of the chromosome

Static Public Member Functions

- static void **recombine** (const **Chromosome** &parent1, const **Chromosome** &parent2, **Chromosome** &offspring1, **Chromosome** &offspring2, double prob, **Random** &rnd)
recombines two parent chromosomes to produce two offspring chromosomes with a given crossover probability. The crossover is applied with probability 'prob', otherwise the offspring are copies of the parents.

4.1.1 Detailed Description

Represents a chromosome in the genetic algorithm.

The chromosome is represented as a vector of unsigned integers, where each integer corresponds to the UID of a city.

The order of the integers in the vector represents the order in which the cities are visited in the TSP route.

The class provides a method to calculate the total length of the route represented by the chromosome.

The class provides methods for mutating the chromosome (swap, shift, permute, invert) and for recombining two chromosomes to produce offspring with a crossover.

The class also provides a method to check if the chromosome is valid.

4.1.2 Constructor & Destructor Documentation

4.1.2.1 Chromosome() [1/3]

```
Chromosome::Chromosome (
    std::vector< unsigned int > possible_UIDs,
    Random & rnd)
```

Constructs a chromosome with a random route based on the provided possible UIDs. The constructor initializes the chromosome with a valid route that includes all the possible UIDs exactly once. The first element of the chromosome is set to the minimum UID to reduce symmetry of the problem.

Parameters

<i>possible_UIDs</i>	a vector of possible UIDs to include in the chromosome
<i>rnd</i>	a random number generator

4.1.2.2 Chromosome() [2/3]

```
Chromosome::Chromosome (
    std::vector< unsigned int > possible_UIDs,
    std::vector< unsigned int > string)
```

Constructs a chromosome with a given route. The constructor checks if the provided route is valid (contains all possible UIDs exactly once and starts with the minimum UID).

Parameters

<i>possible_UIDs</i>	a vector of possible UIDs to include in the chromosome
<i>string</i>	a vector representing the route of cities to visit

4.1.2.3 Chromosome() [3/3]

```
Chromosome::Chromosome (
    const Chromosome & other) [inline]
```

copy constructor

Parameters

<i>other</i>	another chromosome to copy from
--------------	---------------------------------

4.1.3 Member Function Documentation

4.1.3.1 check_valid_string()

```
bool Chromosome::check_valid_string (
```

```
std::vector< unsigned int > possible_UIDs) const
```

checks if the chromosome is valid. A chromosome is valid if it contains all possible UUIDs exactly once and starts with the minimum UUID.

Parameters

<i>possible_UIDs</i>	a vector of possible UUIDs to check against
----------------------	---

Returns

true if the chromosome is valid, false otherwise

4.1.3.2 get_string() [1/2]

```
std::vector< unsigned int > & Chromosome::get_string () [inline], [nodiscard]
```

gets the route of the chromosome

Returns

a vector representing the route of cities to visit

4.1.3.3 get_string() [2/2]

```
std::vector< unsigned int > Chromosome::get_string () const [inline]
```

gets the route represented by the chromosome.

Returns

a vector representing the route of cities to visit

4.1.3.4 L1()

```
double Chromosome::L1 (
    const Cities & map) const [nodiscard]
```

calculates the total distance of the route represented by the chromosome.

Parameters

<i>map</i>	the map containing the city coordinates
------------	---

Returns

the total length of the route, using the L1 distance.

4.1.3.5 mutate()

```
void Chromosome::mutate (
    const double & p_swap,
    const double & p_shift,
    const double & p_permute,
    const double & p_invert,
    Random & rnd)
```

mutates the chromosome by applying various mutation operators with given probabilities Mutation operators include swap, shift, permute, and invert, which are applied sequentially.

Parameters

<i>p_swap</i>	probability of performing a swap mutation
<i>p_shift</i>	probability of performing a shift mutation
<i>p_permute</i>	probability of performing a permute mutation
<i>p_invert</i>	probability of performing an invert mutation
<i>rnd</i>	random number generator

4.1.3.6 operator=()

```
Chromosome & Chromosome::operator= (
    const Chromosome & other) [inline]
```

copy assignment operator

Parameters

<i>other</i>	another chromosome to assign from
--------------	-----------------------------------

Returns

a reference to the assigned chromosome

4.1.3.7 overwrite_string()

```
void Chromosome::overwrite_string (
    std::vector< unsigned int > new_string) [inline]
```

sets the chromosome to a new given route

Parameters

<i>new_string</i>	a vector representing the route of cities to visit
-------------------	--

4.1.3.8 recombine()

```
void Chromosome::recombine (
    const Chromosome & parent1,
    const Chromosome & parent2,
    Chromosome & offspring1,
    Chromosome & offspring2,
    double prob,
    Random & rnd) [static]
```

recombines two parent chromosomes to produce two offspring chromosomes with a given crossover probability. The crossover is applied with probability 'prob', otherwise the offspring are copies of the parents.

Parameters

<i>parent1</i>	the first parent chromosome
<i>parent2</i>	the second parent chromosome
<i>offspring1</i>	the first offspring chromosome
<i>offspring2</i>	the second offspring chromosome
<i>prob</i>	the probability of performing the crossover

<code>rnd</code>	random number generator
------------------	-------------------------

4.1.3.9 save()

```
void Chromosome::save (
    std::ofstream & fout) const
```

saves the route represented by the chromosome to a file. The route is saved as a sequence of UUIDs, one per line.

Parameters

<code>fout</code>	an output file stream to write the route to
-------------------	---

The documentation for this class was generated from the following files:

- include/[chromosome.hpp](#)
- source/[chromosome.cpp](#)

4.2 Cities Class Reference

represents a collection of cities.

```
#include <city.hpp>
```

Public Member Functions

- `const std::vector< unsigned int > & get_possible_UUIDs () const`
gets all the UUIDs of the cities in the collection.
- `const City & get_city (unsigned int idx) const`
gets a city by its index.
- `const double & get_distance (const unsigned int &uid1, const unsigned int &uid2) const`
gets the distance between two cities by their UUIDs.
- `void save_positions (const char *filename)`
saves the positions of the cities in a file. The file should contain two columns of float with x and y coordinates of the cities, nothing else.

Static Public Member Functions

- static `Cities Initialize_Cities_Circle (unsigned int N, double radius, Random &rnd)`
Constructs a collection of cities with random positions on a circle.
- static `Cities Initialize_Cities_Square (unsigned int N, double length, Random &rnd)`
Constructs a collection of cities with random positions inside a square.
- static `Cities Initialize_From_File (const char *filename)`
Construct a collection of cities from the positions contained inside a file.

4.2.1 Detailed Description

represents a collection of cities.

The [Cities](#) class contains a vector of [City](#) objects, and a vector of possible UUIDs (the UUIDs of the cities in the collection).

The [Cities](#) class contains a map that associates each possible UUID to the index of the city in the vector, to optimize data retrieval.

The [Cities](#) class also contains a distance matrix, which contains the precomputed distances between all cities for optimization purposes.

The [Cities](#) class is designed to be immutable.

4.2.2 Member Function Documentation

4.2.2.1 get_city()

```
const City & Cities::get_city (
    unsigned int idx) const
```

gets a city by its index.

Parameters

<i>idx</i>	the index of the city to retrieve.
------------	------------------------------------

Returns

a reference to the requested city instance.

4.2.2.2 get_distance()

```
const double & Cities::get_distance (
    const unsigned int & uid1,
    const unsigned int & uid2) const
```

gets the distance between two cities by their UIDs.

Parameters

<i>uid1</i>	the UID of the first city.
<i>uid2</i>	the UID of the second city.

Returns

the distance between the two cities.

4.2.2.3 Initialize_Cities_Circle()

```
Cities Cities::Initialize_Cities_Circle (
    unsigned int N,
    double radius,
    Random & rnd) [static]
```

Constructs a collection of cities with random positions on a circle.

Parameters

<i>N</i>	the number of cities to generate.
<i>radius</i>	the radius of the circle on which the cities will be generated.
<i>rnd</i>	a random number generator.

Returns

a [Cities](#) instance.

4.2.2.4 Initialize_Cities_Square()

```
Cities Cities::Initialize_Cities_Square (
    unsigned int N,
```

```
double length,  
Random & rnd) [static]
```

Constructs a collection of cities with random positions inside a square.

Parameters

<i>N</i>	the number of cities to generate.
<i>length</i>	the length of the sides of the square inside which the cities will be generated.
<i>rnd</i>	a random number generator.

Returns

a [Cities](#) instance.

4.2.2.5 Initialize_From_File()

```
Cities Cities::Initialize_From_File (  
    const char * filename) [static]
```

Construct a collection of cities from the positions contained inside a file.

Parameters

<i>filename</i>	the name of the file where to load the positions.
-----------------	---

Returns

a [Cities](#) instance.

4.2.2.6 save_positions()

```
void Cities::save_positions (  
    const char * filename)
```

saves the positions of the cities in a file. The file should contain two columns of float with x and y coordinates of the cities, nothing else.

Parameters

<i>filename</i>	the name of the file where to save the positions.
-----------------	---

Note

this method assumes that file exists and the cities are in the correct format

The documentation for this class was generated from the following files:

- include/[city.hpp](#)
- source/[city.cpp](#)

4.3 City Class Reference

represents a unique city.

```
#include <city.hpp>
```

Public Member Functions

- [City](#) (const [Position](#) &pos)
constructor
- unsigned int **get_UID** () const
gets the unique identifier of the city
- [Position](#) **get_pos** () const
gets the position of the city
- double **get_x** () const
gets the x coordinate of the city
- double **get_y** () const
gets the y coordinate of the city

4.3.1 Detailed Description

represents a unique city.

The [City](#) class contains the [Position](#) of the city and a unique identifier (UID).

The UID is generated automatically by the class, and is used to identify the city. Two cities will always have different UIDs, even if they have the same position.

4.3.2 Constructor & Destructor Documentation

4.3.2.1 City()

```
City::City (
    const Position & pos) [inline]
constructor
```

Parameters

<i>pos</i>	the position of the city
------------	--------------------------

The documentation for this class was generated from the following files:

- include/[city.hpp](#)
- source/[city.cpp](#)

4.4 Genetic Class Reference

represents a genetic algorithm for solving the traveling salesman problem.

```
#include <genetic.hpp>
```

Public Member Functions

- [Genetic](#) (const unsigned int &population, const [Cities](#) &map, [Random](#) &rnd)
constructor for the [Genetic](#) class. The probabilities for mutations and crossover are set to default values.
- [Genetic](#) (const unsigned int &population, const [Cities](#) &map, const double &prob_swap, const double &prob_↔_shift, const double &prob_permute, const double &prob_invert, const double &prob_crossover, [Random](#) &rnd)
constructor for the [Genetic](#) class. The probabilities for mutations and crossover are set to the provided values.
- void **new_generation** (const [Cities](#) &map, const double &exponent, [Random](#) &rnd)
creates a new generation of chromosomes. The new generation is found by:
- bool **check_all_chromosomes** (const std::vector< unsigned int > &possible_UIDs) const
checks if all chromosomes in the population are valid.
- void **calculate_fitness_first** (const [Cities](#) &map)

computes the fitness of the first chromosome in the population. Also inserts the fitness value in the correct position in the sorted fitness vector.

- double `average_L1` () const
gets the average L1 distance of the best half of the population.
- unsigned int `best_L1_index` () const
gets the index of the chromosome with the best (lowest) L1 distance in the population.
- double `best_L1` () const
gets the best (lowest) L1 distance in the population.
- `Chromosome` & `get_chromosome` (const unsigned int &idx)
gets a specific chromosome from the population by its index.
- void `save_chromosome` (const unsigned int &idx, std::ofstream &fout) const
saves a specific chromosome from the population to a file.
- void `overwrite_first_chromosome` (const std::vector< unsigned int > &other_string)
overwrites the first chromosome with the new route

4.4.1 Detailed Description

represents a genetic algorithm for solving the traveling salesman problem.

The class manages a population of chromosomes, where each chromosome represents a possible solution to the TSP. The class provides methods for evolving the population through selection, crossover, and mutation. The parameters for mutation and crossover probabilities can be set in the constructor, and are stored as member variables. The fitness of each chromosome is calculated based on the total length of the route it represents, using the L1 distance. The class also provides methods for retrieving the best solution and the average fitness of the best half of the population.

The class is designed to work alongside the `Cities` class, which provides the necessary information about the cities and their distances.

4.4.2 Constructor & Destructor Documentation

4.4.2.1 Genetic() [1/2]

```
Genetic::Genetic (
    const unsigned int & population,
    const Cities & map,
    Random & rnd)
```

constructor for the `Genetic` class. The probabilities for mutations and crossover are set to default values.

Parameters

<i>population</i>	the number of chromosomes in the population.
<i>map</i>	the map containing the city coordinates and distances.
<i>rnd</i>	the random number generator.

4.4.2.2 Genetic() [2/2]

```
Genetic::Genetic (
    const unsigned int & population,
    const Cities & map,
    const double & prob_swap,
    const double & prob_shift,
    const double & prob_permute,
    const double & prob_invert,
    const double & prob_crossover,
    Random & rnd)
```

constructor for the `Genetic` class. The probabilities for mutations and crossover are set to the provided values.

Parameters

<i>population</i>	the number of chromosomes in the population.
<i>map</i>	the map containing the city coordinates and distances.
<i>prob_swap</i>	the probability of performing a swap mutation.
<i>prob_shift</i>	the probability of performing a shift mutation.
<i>prob_permute</i>	the probability of performing a permute mutation.
<i>prob_invert</i>	the probability of performing an invert mutation.
<i>prob_crossover</i>	the probability of performing a crossover.
<i>rnd</i>	the random number generator.

4.4.3 Member Function Documentation**4.4.3.1 average_L1()**

`double Genetic::average_L1 () const`
 gets the average L1 distance of the best half of the population.

Returns

the average L1 distance of the best half of the population.

4.4.3.2 best_L1()

`double Genetic::best_L1 () const`
 gets the best (lowest) L1 distance in the population.

Returns

the best L1 distance in the population.

4.4.3.3 best_L1_index()

`unsigned int Genetic::best_L1_index () const`
 gets the index of the chromosome with the best (lowest) L1 distance in the population.

Returns

the index of the chromosome with the best L1 distance.

4.4.3.4 calculate_fitness_first()

`void Genetic::calculate_fitness_first (`
 `const Cities & map)`

computes the fitness of the first chromosome in the population. Also inserts the fitness value in the correct position in the sorted fitness vector.

Parameters

<i>map</i>	the map containing the city coordinates and distances
------------	---

4.4.3.5 check_all_chromosomes()

```
bool Genetic::check_all_chromosomes (
    const std::vector< unsigned int > & possible_UIDs) const
```

checks if all chromosomes in the population are valid.

Parameters

<i>possible_UIDs</i>	a vector containing the UUIDs of all possible cities.
----------------------	---

Returns

true if all chromosomes are valid, false otherwise.

4.4.3.6 get_chromosome()

```
Chromosome & Genetic::get_chromosome (
    const unsigned int & idx) [inline]
```

gets a specific chromosome from the population by its index.

Parameters

<i>idx</i>	the index of the chromosome to retrieve.
------------	--

Returns

a reference to the requested chromosome.

4.4.3.7 new_generation()

```
void Genetic::new_generation (
    const Cities & map,
    const double & exponent,
    Random & rnd)
```

creates a new generation of chromosomes. The new generation is found by:

1. Sorting the chromosomes based on their fitness (best fitness first).
2. Selecting two random parents from the current population using the selection method discussed below.
3. Recombining the selected parents to create two offspring using the crossover method.
4. Mutating the offspring.
5. Repeating from step 2 until the new population is the same size as the original.
6. Replacing the old population with the new one.
7. Recomputing the fitness of the new population.

The selection method is based on the fitness of the chromosomes. The individuals are selected using the formula:

$$j = \text{populationsize} \cdot \text{floor}(r^p)$$

where r is a random number in the range $[0,1)$ and p is an exponent that can be tuned to increase or decrease the selection pressure. Since the fitness is sorted from best to worst, a higher value of p will increase the selection pressure.

Parameters

<i>map</i>	the map containing the city coordinates and distances.
<i>exponent</i>	the exponent used in the selection method to tune selection pressure.
<i>rnd</i>	the random number generator.

4.4.3.8 overwrite_first_chromosome()

```
void Genetic::overwrite_first_chromosome (
    const std::vector< unsigned int > & other_string)
overwrites the first chromosome with the new route
```

Parameters

<i>other_string</i>	a vector of the new route.
---------------------	----------------------------

The documentation for this class was generated from the following files:

- include/[genetic.hpp](#)
- source/[genetic.cpp](#)

4.5 Position Class Reference

represents a position in a 2D space.

```
#include <position.hpp>
```

Public Member Functions

- **Position** ()
default constructor, initializes the position to (0, 0).
- **Position** (double x, double y)
constructor that initializes the position to the given x and y coordinates.
- **Position** (const **Position** &pos)
copy constructor
- const double & **get_x** () const
gets the x coordinate of the position.
- const double & **get_y** () const
gets the y coordinate of the position.

Static Public Member Functions

- static double **l1** (const **Position** &pos1, const **Position** &pos2)
calculates the l1 distance between two positions.

4.5.1 Detailed Description

represents a position in a 2D space.

The **Position** class has two member variables, `m_x` and `m_y`, which represent the x and y coordinates of the position.

4.5.2 Constructor & Destructor Documentation**4.5.2.1 Position() [1/2]**

```
Position::Position (
    double x,
    double y) [inline]
```


constructor that initializes the position to the given x and y coordinates.

Parameters

<i>x</i>	The x coordinate.
<i>y</i>	The y coordinate.

4.5.2.2 Position() [2/2]

```
Position::Position (
    const Position & pos) [inline]
```

copy constructor

Parameters

<i>pos</i>	Another instance of Position .
------------	--

4.5.3 Member Function Documentation

4.5.3.1 l1()

```
double Position::l1 (
    const Position & pos1,
    const Position & pos2) [inline], [static], [nodiscard]
```

calculates the l1 distance between two positions.

Parameters

<i>pos1</i>	The first position.
<i>pos2</i>	The second position.

Returns

The l1 distance between the two positions.

The l1 distance is calculated as:

$$L_1 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The documentation for this class was generated from the following file:

- [include/position.hpp](#)

4.6 Random Class Reference

A class for generating random numbers.

```
#include <random.h>
```

Public Member Functions

- void **SetRandom** (int *, int, int)
Method to set the seed for the RNG.
- void **SaveSeed** ()
Method to save the seed to a file.
- double **Rannyu** (void)

Method to generate a random number in the range $[0, 1)$.

- double **Rannyu** (double min, double max)

Method to generate a random number in the range $[min, max)$.

- double **Gauss** (double mean, double sigma)

Method to generate a random number with a Gaussian distribution.

4.6.1 Detailed Description

A class for generating random numbers.

The [Random](#) class provides methods for generating random numbers using uniform and Gaussian distributions. It also includes methods for setting the seed and saving it to a file.

Author

D.E. Galli

The documentation for this class was generated from the following files:

- include/[random.h](#)
- source/[random.cpp](#)

Chapter 5

File Documentation

5.1 include/chromosome.hpp File Reference

Stores class [Chromosome](#).

```
#include <unordered_set>
#include "city.hpp"
```

Classes

- class [Chromosome](#)

Represents a chromosome in the genetic algorithm.

5.1.1 Detailed Description

Stores class [Chromosome](#).

5.2 chromosome.hpp

[Go to the documentation of this file.](#)

```
00001
00005
00006 #include <unordered_set>
00007
00008 #include "city.hpp"
00009
00026 class Chromosome
00027 {
00028     std::vector<unsigned int> m_string; // route of cities UIDs to visit
00029
00035     void mutate_swap(const unsigned int &idx1, const unsigned int &idx2, const double &prob, Random
&rnd);
00036
00041     void mutate_swap_all(const double &prob, Random &rnd);
00042
00046     void mutate_swap_init(Random &rnd);
00047
00055     void mutate_shift(const double &prob, Random &rnd);
00056
00062     void mutate_permute(const double &prob, Random &rnd);
00063
00069     void mutate_invert(const double &prob, Random &rnd);
00070
00071     Chromosome() = delete;
00072
00073 public:
00079     Chromosome(std::vector<unsigned int> possible_UIDs, Random &rnd);
00084     Chromosome(std::vector<unsigned int> possible_UIDs, std::vector<unsigned int> string);
00085
00088     Chromosome(const Chromosome &other) : m_string(other.m_string) {};
00092     Chromosome &operator=(const Chromosome &other)
00093     {
00094         m_string = other.m_string;
00095         return *this;
00096     }
```

```

00097
00105     void mutate(const double &p_swap, const double &p_shift, const double &p_permute, const double
&p_invert, Random &rnd);
00106
00115     static void recombine(const Chromosome &parent1, const Chromosome &parent2, Chromosome
&offspring1, Chromosome &offspring2, double prob, Random &rnd);
00116
00121     bool check_valid_string(std::vector<unsigned int> possible_UIDs) const;
00122
00126     [[nodiscard]] double L1(const Cities &map) const;
00127
00130     std::vector<unsigned int> get_string() const { return m_string; }
00131
00135     void save(std::ofstream &fout) const;
00136
00139     void overwrite_string(std::vector<unsigned int> new_string) { m_string = new_string; };
00140
00143     [[nodiscard]] std::vector<unsigned int> &get_string() { return m_string; }
00144 };

```

5.3 include/city.hpp File Reference

Stores the class [City](#), and the class [Cities](#).

```

#include "position.hpp"
#include "random.h"
#include <vector>
#include <algorithm>
#include <fstream>

```

Classes

- class [City](#)
represents a unique city.
- class [Cities](#)
represents a collection of cities.

5.3.1 Detailed Description

Stores the class [City](#), and the class [Cities](#).

5.4 city.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #include "position.hpp"
00007 #include "random.h"
00008 #include <vector>
00009 #include <algorithm>
00010 #include <fstream>
00011
00021 class City
00022 {
00023     static unsigned int s_next_uid; // for uid generator
00024     const Position m_pos;          // position of the city
00025     const unsigned int UID;         // uid of the city
00026
00027 public:
00030     City(const Position &pos) : m_pos(pos), UID(s_next_uid++) {};
00031
00033     unsigned int get_UID() const { return UID; }
00034
00036     Position get_pos() const { return m_pos; }
00038     double get_x() const { return m_pos.get_x(); }
00040     double get_y() const { return m_pos.get_y(); }
00041 };
00042
00058 class Cities
00059 {
00060     const std::vector<City> m_cities; // vector of cities

```

```

00061     const std::vector<unsigned int> m_possible_UIDs;           // vector of cities uids
00062     std::unordered_map<unsigned int, unsigned int> m_uid_to_index; // to optimize data retrieval
00063     std::vector<std::vector<double>> m_distance_matrix;        // to optimize distance calculations
00064
00065     Cities() = delete;
00066
00067     // internal constructor
00068     Cities(const std::vector<City> &cities, const std::vector<unsigned int> &UIDs);
00069
00070 public:
00071     static Cities Initialize_Cities_Circle(unsigned int N, double radius, Random &rnd);
00072
00073     static Cities Initialize_Cities_Square(unsigned int N, double length, Random &rnd);
00074
00075     static Cities Initialize_From_File(const char *filename);
00076
00077     const std::vector<unsigned int> &get_possible_UIDs() const { return m_possible_UIDs; };
00078
00079     const City &get_city(unsigned int idx) const;
00080     const double &get_distance(const unsigned int &uid1, const unsigned int &uid2) const;
00081
00082     void save_positions(const char *filename);
00083 };

```

5.5 include/genetic.hpp File Reference

stores the class [Genetic](#)

```

#include <numeric>
#include <algorithm>
#include "chromosome.hpp"

```

Classes

- class [Genetic](#)

represents a genetic algorithm for solving the traveling salesman problem.

5.5.1 Detailed Description

stores the class [Genetic](#)

5.6 genetic.hpp

[Go to the documentation of this file.](#)

```

00001
00002
00003 #include <numeric>
00004 #include <algorithm>
00005
00006 #include "chromosome.hpp"
00007
00008 class Genetic
00009 {
00010     const unsigned int m_population;
00011
00012     std::vector<Chromosome> m_chromosomes;
00013     std::vector<Chromosome> m_dummy_chromosomes;
00014     std::vector<double> m_fitness;
00015     std::vector<double> m_dummy_fitness;
00016
00017     const double m_p_m_swap = 0.05;
00018     const double m_p_m_shift = 0.1;
00019     const double m_p_m_permute = 0.1;
00020     const double m_p_m_invert = 0.05;
00021     const double m_p_crossover = 0.6;
00022
00023     void calculate_fitness(const Cities &map);
00024     void sort_chromosomes();
00025
00026 public:
00027     Genetic(const unsigned int &population, const Cities &map, Random &rnd);
00028     Genetic(const unsigned int &population, const Cities &map, const double &prob_swap, const double
&prob_shift, const double &prob_permute, const double &prob_invert, const double &prob_crossover,
Random &rnd);

```

```

00072
00092     void new_generation(const Cities &map, const double &exponent, Random &rnd);
00093
00097     bool check_all_chromosomes(const std::vector<unsigned int> &possible_UIDs) const;
00098
00102     void calculate_fitness_first(const Cities &map);
00103
00106     double average_L1() const;
00107
00110     unsigned int best_L1_index() const;
00113     double best_L1() const;
00114
00118     Chromosome &get_chromosome(const unsigned int &idx) { return m_chromosomes[idx]; }
00119
00121     void save_chromosome(const unsigned int &idx, std::ofstream &fout) const;
00122
00125     void overwrite_first_chromosome(const std::vector<unsigned int> &other_string);
00126 };

```

5.7 include/position.hpp File Reference

stores the [Position](#) class.

```

#include <iostream>
#include <cmath>

```

Classes

- class [Position](#)
represents a position in a 2D space.

5.7.1 Detailed Description

stores the [Position](#) class.

5.8 position.hpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #include <iostream>
00007 #include <cmath>
00008
00015 class Position
00016 {
00017     double m_x, m_y;
00018
00019 public:
00021     Position() : m_x(0), m_y(0) {};
00025     Position(double x, double y) : m_x(x), m_y(y) {};
00028     Position(const Position &pos) : m_x(pos.m_x), m_y(pos.m_y) {};
00029
00038     [[nodiscard]] static double l1(const Position &pos1, const Position &pos2)
00039     {
00040         return std::sqrt(std::pow(pos2.m_x - pos1.m_x, 2) + std::pow(pos2.m_y - pos1.m_y, 2));
00041     };
00042
00044     const double &get_x() const { return m_x; }
00046     const double &get_y() const { return m_y; }
00047 };

```

5.9 include/random.h File Reference

This header file defines the [Random](#) class, which contains functions for generating random numbers.

Classes

- class [Random](#)
A class for generating random numbers.

5.9.1 Detailed Description

This header file defines the `Random` class, which contains functions for generating random numbers.

Author

D.E. Galli

5.10 random.h

[Go to the documentation of this file.](#)

```
00001 /*****  
00002 *****/  
00003  
00004      _/_/_/_/_/_/_ Numerical Simulation Laboratory  
00005      _/_/_/_/_/_ Physics Department  
00006      _/_/_/_/_/_ Universita' degli Studi di Milano  
00007 _/_/_/_/_/_ Prof. D.E. Galli  
00008 email: Davide.Galli@unimi.it  
00009 *****  
00010 *****/  
00011  
00012  
00013  
00014 #ifndef __Random__  
00015 #define __Random__  
00016  
00017  
00018 class Random  
00019 {  
00020  
00021 private:  
00022     int m1, m2, m3, m4, l1, l2, l3, l4, n1, n2, n3, n4;  
00023  
00024 protected:  
00025 public:  
00026     // Default constructor  
00027     Random();  
00028     // Destructor  
00029     ~Random();  
00030     void SetRandom(int *, int, int);  
00031     void SaveSeed();  
00032     double Rannyu(void);  
00033     double Rannyu(double min, double max);  
00034     double Gauss(double mean, double sigma);  
00035 };  
00036  
00037 #endif // __Random__  
00038  
00039  
00040 /*****  
00041 *****/  
00042  
00043      _/_/_/_/_/_ Numerical Simulation Laboratory  
00044      _/_/_/_/_/_ Physics Department  
00045      _/_/_/_/_/_ Universita' degli Studi di Milano  
00046      _/_/_/_/_/_ Prof. D.E. Galli  
00047 _/_/_/_/_/_ email: Davide.Galli@unimi.it  
00048 *****  
00049 *****
```

5.11 source/chromosome.cpp File Reference

Implements the modules of the class [Chromosome](#).

```
#include "chromosome.hpp"
```

5.11.1 Detailed Description

Implements the modules of the class [Chromosome](#).

5.12 source/city.cpp File Reference

Implements the modules of the class `City`, and the class `Cities`.

```
#include "city.hpp"
```

5.12.1 Detailed Description

Implements the modules of the class [City](#), and the class [Cities](#).

5.13 source/genetic.cpp File Reference

Implementation of the [Genetic](#) class methods.

```
#include "genetic.hpp"
```

5.13.1 Detailed Description

Implementation of the [Genetic](#) class methods.

5.14 source/random.cpp File Reference

Implements the modules of the class [Random](#).

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <cstdlib>
#include "random.h"
```

5.14.1 Detailed Description

Implements the modules of the class [Random](#).

Author

D.E. Galli

Index

- average_L1
 - Genetic, [16](#)
- best_L1
 - Genetic, [16](#)
- best_L1_index
 - Genetic, [16](#)
- calculate_fitness_first
 - Genetic, [16](#)
- check_all_chromosomes
 - Genetic, [16](#)
- check_valid_string
 - Chromosome, [8](#)
- Chromosome, [7](#)
 - check_valid_string, [8](#)
 - Chromosome, [8](#)
 - get_string, [9](#)
 - L1, [9](#)
 - mutate, [9](#)
 - operator=, [10](#)
 - overwrite_string, [10](#)
 - recombine, [10](#)
 - save, [11](#)
- Cities, [11](#)
 - get_city, [12](#)
 - get_distance, [12](#)
 - Initialize_Cities_Circle, [12](#)
 - Initialize_Cities_Square, [12](#)
 - Initialize_From_File, [13](#)
 - save_positions, [13](#)
- City, [13](#)
 - City, [14](#)
- Genetic, [14](#)
 - average_L1, [16](#)
 - best_L1, [16](#)
 - best_L1_index, [16](#)
 - calculate_fitness_first, [16](#)
 - check_all_chromosomes, [16](#)
 - Genetic, [15](#)
 - get_chromosome, [17](#)
 - new_generation, [17](#)
 - overwrite_first_chromosome, [18](#)
- get_chromosome
 - Genetic, [17](#)
- get_city
 - Cities, [12](#)
- get_distance
 - Cities, [12](#)
- get_string
 - Chromosome, [9](#)
- include/chromosome.hpp, [21](#)
- include/city.hpp, [22](#)
- include/genetic.hpp, [23](#)
- include/position.hpp, [24](#)
- include/random.h, [24](#), [25](#)
- Initialize_Cities_Circle
 - Cities, [12](#)
- Initialize_Cities_Square
 - Cities, [12](#)
- Initialize_From_File
 - Cities, [13](#)
- L1
 - Chromosome, [9](#)
- l1
 - Position, [19](#)
- mutate
 - Chromosome, [9](#)
- My Genetic Algorithm for Travelling Salesman Problem,
 - [1](#)
- new_generation
 - Genetic, [17](#)
- operator=
 - Chromosome, [10](#)
- overwrite_first_chromosome
 - Genetic, [18](#)
- overwrite_string
 - Chromosome, [10](#)
- Position, [18](#)
 - l1, [19](#)
 - Position, [18](#), [19](#)
- Random, [19](#)
- recombine
 - Chromosome, [10](#)
- save
 - Chromosome, [11](#)
- save_positions
 - Cities, [13](#)
- source/chromosome.cpp, [25](#)
- source/city.cpp, [25](#)
- source/genetic.cpp, [26](#)
- source/random.cpp, [26](#)